# Equivalent Prompt Formats at a Correctness Ceiling: A Bounded Code-Generation Study

Altan Esmer
Scientific Methods course project
Preliminary draft for author review, 23 October 2026

***Abstract*—Prompt sensitivity concerns changes in model behavior when a prompt is varied. A functional-correctness experiment can detect such changes only when its tasks and tests expose differences. This exploratory study compares prose, numbered-list, and XML presentations of equivalent programming requirements. Twelve selected public competition problems were presented twice in each format, producing 72 first responses with the requested model `gpt-6.1-sol` and medium reasoning. Tasks, prompts, tests, allocation, and analysis were frozen before collection. Isolated generation excluded configured access to local references and tests. Every program passed its retained suite, giving 13,068 successful case checks across 72 programs. All task-level format contrasts and both equal-task mean contrasts were zero. These observations establish a finite-sample correctness ceiling, not statistical equivalence or general prompt insensitivity. The study documents a reproducible comparison and the remaining measurement limits, including public task exposure, small retained inputs, two responses per cell, and incomplete provider observability.**

***Index Terms*—prompt sensitivity, code generation, prompt formatting, functional correctness, ceiling effect**

## I. Introduction

When requirements remain the same, a model user may expect presentation choices to leave the quality of generated code unchanged. This expectation is testable, but the answer depends on which prompts, tasks, models, and outcomes are compared. Sclar et al. report substantial accuracy differences under formatting variations in their few-shot evaluations of open models [1, p. 1]. That result motivates checking multiple formats; it does not specify the effect expected for a different model or for executable programs.

This paper studies a bounded question: *How do prose, numbered-list, and XML presentations of equivalent programming requirements affect all-case pass rates for `gpt-6.1-sol` with medium reasoning on twelve selected public programming tasks?* The goal is to describe observed correctness under a fixed intervention. It is not to optimize prompts or to rank models.

The contribution is an auditable comparison with separate collection and grading and explicit reference isolation. All 72 programs passed their retained tests. We report this ceiling without replacing tasks or outcomes after collection, and examine the resulting measurement limits.

## II. Related work

### A. Prompt interventions differ

Demonstration order is one source of variation. Lu et al. show that changing the order of few-shot examples can change classification performance [2, p. 8086]. Zhao et al. report instability across prompt formats, examples, and their order, and reduce variance with contextual calibration [3, PDF p. 1]. Their calibration estimates answer bias in few-shot prediction. It differs from this project's earlier phases called calibration, which screened task difficulty.

Our intervention preserves the ordered requirement blocks and adds no demonstrations. It changes their rendering. Accordingly, evidence about example order provides related context rather than a prediction of the present format contrasts.

### B. Sensitivity depends on the outcome

Hua et al. identify heuristic evaluation methods as a contributor to apparent prompt sensitivity in their evaluated natural-language benchmarks [4, p. 19889]. Their finding motivates separating properties of responses from properties of scoring. It does not establish that every observed prompt effect is an evaluation artifact.

ProSA studies instance-level prompt variation and reports differences across models and datasets [5, p. 1950]. POSIX instead quantifies changes in response log-likelihood under prompt variation [6, p. 14550]. Sheng Lu et al. use sensitivity as a proxy for model performance in their evaluated setting [7, p. 5833]. These operational definitions differ from the all-case functional outcome used here. Different source programs can pass the same fixtures while differing in likelihood, style, efficiency, or behavior on untested inputs.

### C. Broader robustness and mitigation

Qiang et al. study input perturbations in intent classification and slot filling and propose prompt perturbation consistency learning [8, p. 1357]. Cao et al. introduce an evaluation based on semantically equivalent case-level queries [9, p. 1]. Fu and Barez propose Latent Adversarial Paraphrasing, which optimizes latent perturbations together with model performance [10, p. 31305].

These interventions and outcomes differ from our fixed-renderer comparison of executable correctness. Their results motivate measurement but do not supply a common effect size to pool with our observations.

## III. Data and methods

### A. Task source and preparation

We used the test split of the BAAI/TACO dataset, whose official record describes code generation from natural-language specifications [11, dataset card]. The frozen dataset revision is `d593ed0a2becbbc952230bb89be09189bf1056dc`. The source problems were authored outside this project's AI-assisted task construction. External authorship does not imply that the problems were absent from model training.

The final sample contains six tasks labeled MEDIUM and six labeled HARD by the source. Eligible tasks were ranked deterministically by a seeded hash of their external identifier, with the first six per difficulty retained after the documented filters. The filters required standard input/output, sufficient supplied cases, and a statically admissible Python standard-library reference. They excluded interactive or special-judge behavior, floating-point output comparisons, damaged statements, and insufficient reuse provenance. Difficulty labels provided source-defined diversity; they did not measure difficulty for the target model.

Preparation identified statement and fixture defects. Whole-task replacements followed the recorded ranking and documented rules before target-model observations. One semantic contradiction in task 0778 was proved from its rules before its replacement. For the final twelve tasks, input-domain and expected-output-shape checks retained 2,178 of 2,339 supplied cases and recorded 161 exclusions. Every original case, retained index, exclusion reason, and input/output hash remains available. Expected answers were not rewritten to make references pass. Supplied references passed the retained suites before collection.

This process produces a filtered convenience sample with curated test subsets, not an official TACO benchmark score. No target-model calibration gate selected the final tasks, and mixed outcomes were not required for inclusion.

### B. Equivalent format intervention

The canonical contents comprised four fixed implementation clauses and the original question paragraphs in their original order. Format A separated instruction clauses with blank lines and left the question text unchanged. Format B numbered the clauses and question paragraphs. Format C used instruction and paragraph elements with reversible XML escaping. All three included the same outer `<problem>` wrapper. Thus A is the prose renderer within a shared wrapper, rather than a prompt containing no markup.

Automated recovery checks reconstructed the same instruction and question contents from all variants. The clauses required Python 3.12, the standard library, stdin/stdout behavior, and runnable source without explanation or Markdown fences. No variant added hints, demonstrations, role instructions, or a reasoning request. Table I shows one actual clause in each format. Syntax and token-length consequences are part of the treatment; their individual effects are not separated.

TABLE I
Actual instruction excerpt. The remaining clauses and full problem are omitted here, not in the experiment.

| Format | Rendering of the first clause |
|---|---|
| A | `Solve the programming problem below.` |
| B | `1. Solve the programming problem below.` |
| C | `<instruction>Solve the programming problem below.</instruction>` |

### C. Allocation and isolated collection

The design comprised twelve tasks, three formats, and two fresh responses per task and format. Each of the 24 task/repetition blocks contained A, B, and C once. All six possible format orders occurred four times. Blocks and orders were randomized using schedule seed 20261022 and frozen before dispatch. This seed controls allocation, not model sampling.

The client explicitly requested `gpt-6.1-sol` with medium reasoning, disabled fallback, and exposed no execution or browsing tools. Every response used a distinct fresh thread. Generation ran in an unprivileged container without repository, reference, test, prior-answer, home-directory, credential, or Docker-socket mounts. The container had no direct external network route; a restricted trusted relay provided its model-service route. Credentials remained outside generation. Offline synthetic checks tested this boundary.

A live preflight on 23 October 2026 used a synthetic marker unrelated to the study tasks and passed before collection. Dispatch records were checked for the frozen prompts and settings. The backend reported the requested model alias. These records do not independently attest to provider weights, model revision, or internal reasoning behavior. Temperature and model sampling seed were not controlled.

The first final answer from each scheduled trial was retained without feedback, repair, follow-up, or answer selection. Collection closed before grading began. All 72 generations completed once in frozen order. Isolation limits configured retrieval of reference artifacts; it cannot remove pretrained knowledge or establish that public tasks are unfamiliar.

### D. Execution and analysis

The primary outcome was whether a returned program passed every retained case for its task. The frozen grader treated the raw answer as Python source, without stripping fences or repairing code. It compared whitespace-separated output tokens exactly, with no numeric tolerance, case folding, or deletion of extra output.

Each case ran in a fresh Python 3.12 container with one CPU, 512 MiB memory, a 64-process limit, read-only code/input mounts, and no network. The ten-second wall limit began after container creation and included start/attach overhead. Standard output and standard error each had a 1 MiB limit. Expected outputs stayed on the host. Program errors

TABLE II
ALL-CASE SUCCESSES BY TASK AND FORMAT. M AND H ARE THE DATASET DIFFICULTY LABELS. CASES ARE RETAINED FIXTURES PER PROGRAM.

| Task | Label | Cases | A | B | C |
|---|---|---|---|---|---|
| 0074 | M | 143 | 2/2 | 2/2 | 2/2 |
| 0152 | H | 18 | 2/2 | 2/2 | 2/2 |
| 0250 | M | 254 | 2/2 | 2/2 | 2/2 |
| 0320 | M | 237 | 2/2 | 2/2 | 2/2 |
| 0339 | H | 225 | 2/2 | 2/2 | 2/2 |
| 0402 | M | 179 | 2/2 | 2/2 | 2/2 |
| 0466 | H | 135 | 2/2 | 2/2 | 2/2 |
| 0472 | M | 160 | 2/2 | 2/2 | 2/2 |
| 0581 | H | 205 | 2/2 | 2/2 | 2/2 |
| 0760 | M | 334 | 2/2 | 2/2 | 2/2 |
| 0799 | H | 50 | 2/2 | 2/2 | 2/2 |
| 0979 | H | 238 | 2/2 | 2/2 | 2/2 |
| Total | | 2,178 | 24/24 | 24/24 | 24/24 |

and timeouts counted as unsuccessful responses; infrastructure failures and missing responses were separate incomplete outcomes.

Let $Y_{tfr}$ indicate all-case success for task $t$, format $f$, and repetition $r$. For each task and format, the reported rate is

$$\widehat{p}_{tf} = \frac{Y_{tf1} + Y_{tf2}}{2}. \tag{1}$$

The task-level contrasts are $\widehat{p}_{tB} - \widehat{p}_{tA}$ and $\widehat{p}_{tC} - \widehat{p}_{tA}$. Each mean contrast weights the twelve tasks equally, for example

$$\widehat{\Delta}_{BA} = \frac{1}{12} \sum_{t=1}^{12} (\widehat{p}_{tB} - \widehat{p}_{tA}). \tag{2}$$

The complete-design contrasts required all 72 outcomes to be evaluable. No confirmatory significance or equivalence test was prespecified. Case checks were not treated as independent model responses.

## IV. RESULTS

All scheduled responses had evaluable outcomes. Each program passed every retained case, with no execution or infrastructure errors. Table II reports all task/format cells. Formats A, B, and C each had 24 successes among 24 programs. Every cell was 2/2, every task-level contrast was zero, and both equal-task mean contrasts were zero.

Across the twelve tasks, 2,178 retained fixtures were applied to six programs per task, yielding 13,068 successful case checks. This count describes repeated checks of 72 programs. It is not a sample of 13,068 independent model outputs.

The collection and grading records bind answers to prompt hashes, thread identifiers, image identities, retained source-case indices, and input/output hashes. Verification confirmed the 196-artifact freeze before grading. A separately scoped reviewer reconciled the saved collection, grading, and analysis records. Review used one model family and supplies artifact checking rather than independent provider attestation.

## V. DISCUSSION AND LIMITATIONS

### A. What the ceiling establishes

No all-case success difference was observed among the tested renderers, tasks, and first responses. The data provide no preferred format under the finite retained suites and recorded conditions.

The ceiling leaves relative failure probabilities poorly characterized. Equal scores do not establish statistical equivalence, a universally small effect, or general prompt insensitivity. No equivalence margin or power calculation was specified. We report descriptive contrasts without constructing confidence intervals from the case-check count.

External authorship and restricted generation reduce dependence on project-generated contracts and configured local reference access. They do not explain the ceiling. Strong performance, familiar public tasks, limited fixtures, and the chosen intervention remain plausible explanations that this design cannot separate.

### B. Validity and generalization

The twelve tasks are a filtered convenience sample. Standard input/output and reference eligibility exclude other programming work. Two repetitions per cell provide little information about variability. Fresh threads prevent conversational carryover but do not prove independent sampling inside a shared provider service.

Public questions and solutions could have appeared in training. The model's training cutoff, weights, revision, and some sampling settings are unavailable. The observed alias and requested effort are the available configuration evidence; results should not be generalized to all systems using a similar name.

Retained fixtures also cover only part of the declared input space. For example, task 0339 has retained arrays of at most 30 elements despite a statement limit of 500,000. Task 0760 has at most 20 despite a limit of 100,000. Passing these suites does not establish full-constraint correctness or runtime quality. A program with unsuitable asymptotic cost may pass. The frozen study added no large-input stress suite.

Scoring measures exact required output under the saved contract. It does not measure maintainability, explanation quality, likelihood sensitivity, or all semantic properties of the generated programs. References, validators, protocol preparation, and review were AI-assisted within one model family, so correlated errors remain possible. Recorded source-defect checks and independent scopes reduce particular risks without providing a complete correctness proof.

### C. Earlier phases and next research

Two earlier custom-task pilots each produced 45/45 functional passes. A first baseline task calibration produced 18/18, and a composite-task calibration produced 30/30. No family met the qualification rule for the conditional third pilot, which was not launched. A later audit applied stronger tests to the same 30 saved composite answers and produced 4,770/4,770 successful checks without generating new responses. These

phases informed this design but used different tasks, interventions, or controls. They are historical context and are not pooled with the present 72 programs.

A future experiment should freeze a fresh task source, failure-exposing tests, a sampling rationale, and an analysis plan before collecting responses. It should establish what effect size matters and preserve the task grouping in its uncertainty analysis. New outcomes found by inspecting these saved answers would be exploratory. They should not replace this study's frozen primary endpoint as confirmatory evidence.

## VI. Conclusion

A separately frozen comparison of three equivalent renderers produced 72/72 all-case successes on twelve selected public tasks. Both mean contrasts were zero. The result is an observed correctness ceiling. It supports an auditable methods account and a narrow descriptive finding while leaving broader prompt sensitivity unresolved.

## AI assistance and reproducibility

AI assistants contributed to earlier custom tasks, study design, selection and validation tools, prompt renderers, isolation and grading software, model-response collection, result reconciliation, literature searches, and this manuscript draft. The twelve source problem statements were externally authored. That distinction does not make study construction or verification independent of AI assistance.

For this paper, an assistant checked ten proceedings sources, selected evidence passages, assembled highlighted screenshots, and drafted the outline and LaTeX. Separately scoped agents checked source claims, methods, and results within one model family. This is not independent human scholarly review. The author must review the sources, code, conclusions, and final text; this preliminary draft does not assert that this personal review has occurred.

The public repository is https://github.com/AltanEsmer/ScM. The reported study is in `studies/format-study1`; its frozen protocol is preserved as written before collection, and completed outcomes are recorded separately. Data-source attribution, original case records, exclusions, raw answers, grading records, and analysis code remain available. Dataset and upstream problem rights remain with their respective owners; reuse notices are preserved in the source directory. The citation appendix supplies evidence for every external citation occurrence in this draft.

## References


[1] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying language models'sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting," in *International Conference on Learning Representations*, B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, Eds., vol. 2024, 2024, pp. 25 055–25 083. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/file/6c0e99d736da621403018ca7b32b1a4d-Paper-Conference.pdf

[2] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, "Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 8086–8098. [Online]. Available: https://aclanthology.org/2022.acl-long.556/

[3] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 12 697–12 706. [Online]. Available: https://proceedings.mlr.press/v139/zhao21c.html

[4] A. Hua, K. Tang, C. Gu, J. Gu, E. Wong, and Y. Qin, "Flaw or artifact? rethinking prompt sensitivity in evaluating LLMs," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 19 889–19 899. [Online]. Available: https://aclanthology.org/2025.emnlp-main.1006/

[5] J. Zhuo, S. Zhang, X. Fang, H. Duan, D. Lin, and K. Chen, "ProSA: Assessing and understanding the prompt sensitivity of LLMs," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 1950–1976. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.108/

[6] A. Chatterjee, H. S. V. N. S. K. Renduchintala, S. Bhatia, and T. Chakraborty, "POSIX: A prompt sensitivity index for large language models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 14 550–14 565. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.852/

[7] S. Lu, H. Schuff, and I. Gurevych, "How are prompts different in terms of sensitivity?" in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 5833–5856. [Online]. Available: https://aclanthology.org/2024.naacl-long.325/

[8] Y. Qiang, S. Nandi, N. Mehrabi, G. Ver Steeg, A. Kumar, A. Rumshisky, and A. Galstyan, "Prompt perturbation consistency learning for robust language models," in *Findings of the Association for Computational Linguistics: EACL 2024*, Y. Graham and M. Purver, Eds. St. Julian's, Malta: Association for Computational Linguistics, Mar. 2024, pp. 1357–1370. [Online]. Available: https://aclanthology.org/2024.findings-eacl.91/

[9] B. Cao, D. Cai, Z. Zhang, Y. Zou, and W. Lam, "On the worst prompt performance of large language models," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 69 022–69 042. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/7fa5a377b7ffabcce43cd00231bb3f9c-Paper-Conference.pdf

[10] T. Fu and F. Barez, "Same question, different words: A latent adversarial framework for prompt robustness," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 31 305–31 319. [Online]. Available: https://aclanthology.org/2025.emnlp-main.1595/

[11] Beijing Academy of Artificial Intelligence, "BAAI/TACO: Official dataset record," Hugging Face dataset card, accessed 23 October 2026. [Online]. Available: https://huggingface.co/datasets/BAAI/TACO/blob/d593ed0a2becbbc952230bb89be09189bf1056dc/README.md

## APPENDIX: CITATION EVIDENCE

This appendix is separate from the main paper's page count. The main paper, including references, ends on page 4. Each citation occurs once. Evidence IDs persist across edits; bracketed numbers match the bibliography in this draft. Publisher article pages are reproduced with locally applied yellow highlights and source provenance. They support the specific cited claims, not broader conclusions about our model. The dataset record is an additional first-party software/data citation and is not counted among the ten peer-reviewed papers.

| Reference | Occurrence | Source | Evidence location |
|---|---|---|---|
| [1] | C01 | Sclar et al., ICLR 2024 | Publisher PDF p. 1 |
| [2] | C02 | Yao Lu et al., ACL 2022 | Publisher p. 8086 |
| [3] | C08 | Zhao et al., ICML 2021 | Publisher PDF p. 1 |
| [4] | C03 | Hua et al., EMNLP 2025 | Publisher p. 19889 |
| [5] | C04 | Zhuo et al., Findings 2024 | Publisher p. 1950 |
| [6] | C05 | Chatterjee et al., Findings 2024 | Publisher p. 14550 |
| [7] | C07 | Sheng Lu et al., NAACL 2024 | Publisher p. 5833 |
| [8] | C06 | Qiang et al., Findings 2024 | Publisher p. 1357 |
| [9] | C09 | Cao et al., NeurIPS 2024 | Publisher PDF p. 1 |
| [10] | C10 | Fu and Barez, EMNLP 2025 | Publisher p. 31305 |
| [11] | C11 | BAAI/TACO | Official pinned dataset card |

The following ten article-evidence pages are organized by persistent source ID. Their reference tags identify the bibliography entries even when physical page order differs from citation order. The last evidence page documents the dataset record. Full publisher URLs and exact highlight fragments are also retained in the repository manifests.

**[1] C01 / S01 - publisher passage**

Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I learned to start worrying about prompt formatting
Abstract. PDF page 1. Printed page 1.
https://proceedings.iclr.cc/paper_files/paper/2024/file/6c0e99d736da621403018ca7b32b1a4d-Paper-Conference.pdf
Supports: Formatting choices produced accuracy variation in the reported few-shot setting.



# QUANTIFYING LANGUAGE MODELS' SENSITIVITY TO SPURIOUS FEATURES IN PROMPT DESIGN *or: How I learned to start worrying about prompt formatting*

**Melanie Sclar**[1] **Yejin Choi**[1,2] **Yulia Tsvetkov**[1] **Alane Suhr**[3]
[1]Paul G. Allen School of Computer Science & Engineering, University of Washington
[2]Allen Institute for Artificial Intelligence [3]University of California, Berkeley
msclar@cs.washington.edu

## ABSTRACT

As large language models (LLMs) are adopted as a fundamental component of language technologies, it is crucial to accurately characterize their performance. Because choices in prompt design can strongly influence model behavior, this design process is critical in effectively using any modern pre-trained generative language model. In this work, we focus on LLM sensitivity to a quintessential class of meaning-preserving design choices: prompt formatting. We find that several widely used open-source LLMs are extremely sensitive to subtle changes in prompt formatting in few-shot settings, with performance differences of up to 76 accuracy points when evaluated using LLaMA-2-13B. Sensitivity remains even when increasing model size, the number of few-shot examples, or performing instruction tuning. Our analysis suggests that work evaluating LLMs with prompting-based methods would benefit from reporting a range of performance across plausible prompt formats, instead of the currently-standard practice of reporting performance on a single format. We also show that format performance only weakly correlates between models, which puts into question the methodological validity of comparing models with an arbitrarily chosen, fixed prompt format. To facilitate systematic analysis we propose FORMATSPREAD, an algorithm that rapidly evaluates a sampled set of plausible prompt formats for a given task, and reports the interval of expected performance without accessing model weights[1]. Furthermore, we present a suite of analyses that characterize the nature of this sensitivity, including exploring the influence of particular atomic perturbations and the internal representation of particular formats.

## 1 INTRODUCTION

As the capabilities of LLMs have rapidly improved, their sensitivity to input prompt features has been used to optimize performance via prompt engineering (White et al., 2023). However, there has been little work in characterizing this sensitivity, especially to seemingly innocuous feature choices that preserve prompt meaning and intent. In this work, we analyze the sensitivity of widely used, open-source LLMs to a class of features that should not influence a prompt's interpretation: formatting choices. We find that pre-trained LLMs are sensitive to these choices in unpredictable ways, with accuracy varying in up to 76 points for LLaMA-2-13B between equivalent formats, and $\sim$10 accuracy points on average across 50+ tasks and several models. We also show that this variance is not eliminated by adding few-shot examples, increasing model size, or instruction tuning.

Designing prompt templates is a critical part of effectively using a pre-trained language model. This design process includes making choices about wording, choosing few-shot examples for in-context learning, and making decisions about seemingly trivial features like formatting. This process, and often even the resulting templates, is rarely reported or discussed in research papers, under the assumption that performance variance across these choices is insignificant compared to variance across data points or models. However, some anecdotal evidence points to formatting choices actually having a significant influence on model behavior (Aghajanyan, 2023). In some cases, researchers report a limited number of manually generated formats to show that scaling trends hold despite perfor-

[1]FORMATSPREAD's code can be found at https://github.com/msclar/formatspread.

**[2] C02 / S02 - publisher passage**

Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity
Abstract. PDF page 1. Printed page 8086.
https://aclanthology.org/2022.acl-long.556.pdf
Supports: Changing demonstration order changed performance in the evaluated few-shot setting.

# Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity

**Yao Lu**[†] **Max Bartolo**[†] **Alastair Moore**[‡] **Sebastian Riedel**[†] **Pontus Stenetorp**[†]
[†]University College London [‡]Mishcon de Reya LLP
{yao.lu,m.bartolo,s.riedel,p.stenetorp}@cs.ucl.ac.uk
alastair.moore@mishcon.com

## Abstract

When primed with only a handful of training samples, very large, pretrained language models such as GPT-3 have shown competitive results when compared to fully-supervised, fine-tuned, large, pretrained language models. We demonstrate that the order in which the samples are provided can make the difference between near state-of-the-art and random guess performance: essentially some permutations are "fantastic" and some not. We analyse this phenomenon in detail, establishing that: it is present across model sizes (even for the largest current models), it is not related to a specific subset of samples, and that a given good permutation for one model is not transferable to another. While one could use a development set to determine which permutations are performant, this would deviate from the true few-shot setting as it requires additional annotated data. Instead, we use the generative nature of language models to construct an artificial development set and based on entropy statistics of the candidate permutations on this set, we identify performant prompts. Our method yields a 13% relative improvement for GPT-family models across eleven different established text classification tasks.

## 1 Introduction

Large pretrained language models (PLMs, Devlin et al., 2019; Peters et al., 2018; Raffel et al., 2020; Liu et al., 2019; Yang et al., 2019; Radford et al., 2019) have shown remarkable performance when conditioned with an appropriate textual context (Petroni et al., 2019, 2020; Jiang et al., 2020; Shin et al., 2020; Davison et al., 2019). For example, when conditioned on a long document and a "TL;DR:" token, they can generate a summary of said document, and when provided a partial question ("The theory of relativity was developed by __"), they can generate the correct answer. Perhaps most strikingly, when primed with a context consisting of very few training examples, they produce text classification results that can match those of fully supervised models. This type of few shot setting, is commonly referred to as "In-context Learning" (Brown et al., 2020).

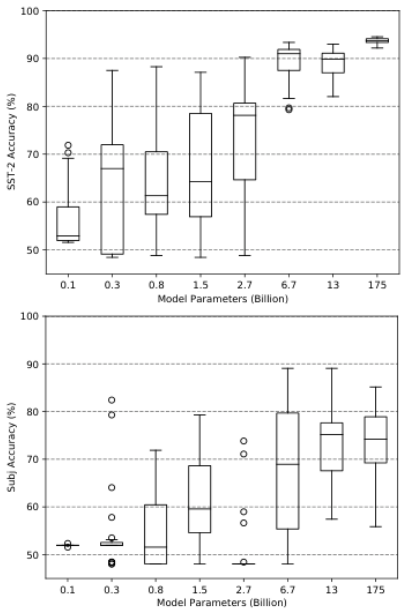



Figure 1: Four-shot performance for 24 different sample orders across different sizes of GPT-family models (GPT-2 and GPT-3) for the SST-2 and Subj datasets.

A core component of in-context learning is the text-based prompt that serves as the context. Composing a prompt requires: (i) text linearisation using a template; and (ii) training sample concatenation (See Table 1 for an example). It has been established that the structure of the template has a large impact on performance (Shin et al., 2020; Gao et al., 2020; Schick and Schütze, 2020; Jiang et al., 2020). However, to the best of our knowledge, no work has studied the effect of the sample ordering on In-context Learning performance.

Perhaps counter-intuitively, we find that the right sample order can make as much of a difference as



*Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*
*Volume 1: Long Papers*, pages 8086 - 8098
May 22-27, 2022 ©2022 Association for Computational Linguistics

**[4] C03 / S03 - publisher passage**

Flaw or Artifact? Rethinking Prompt Sensitivity in Evaluating LLMs
Abstract. PDF page 1. Printed page 19889.
https://aclanthology.org/2025.emnlp-main.1006.pdf
Supports: Heuristic scoring contributed to measured prompt sensitivity in the evaluated benchmarks.

# Flaw or Artifact? Rethinking Prompt Sensitivity in Evaluating LLMs

**Andong Hua[1]*, Kenan Tang[1]*, Chenhe Gu[2], Jindong Gu[3], Eric Wong[4], Yao Qin[1]**
[1] UC Santa Barbara, [2] UC Irvine, [3] University of Oxford, [4] University of Pennsylvania
dongx1997@ucsb.edu, yaoqin@ucsb.edu

## Abstract

Prompt sensitivity, referring to the phenomenon where paraphrasing (i.e., repeating something written or spoken using different words) leads to significant changes in large language model (LLM) performance, has been widely accepted as a core limitation of LLMs. In this work, we revisit this issue and ask: Is the widely reported high prompt sensitivity truly an inherent weakness of LLMs, or is it largely an artifact of evaluation processes? To answer this question, we systematically evaluate 7 LLMs (e.g., GPT and Gemini family) across 6 benchmarks, including both multiple-choice and open-ended tasks on 12 diverse prompt templates. We find that much of the prompt sensitivity stems from heuristic evaluation methods, including log-likelihood scoring and rigid answer matching, which often overlook semantically correct responses expressed through alternative phrasings, such as synonyms or paraphrases. When we adopt LLM-as-a-Judge evaluations, we observe a substantial reduction in performance variance and a consistently higher correlation in model rankings across prompts. Our findings suggest that modern LLMs are more robust to prompt templates than previously believed, and that prompt sensitivity may be more an artifact of evaluation than a flaw in the models.

## 1 Introduction

Large Language Models (LLMs) have achieved remarkable success across a wide range of tasks (Hua et al., 2024; Liévin et al., 2024). Moreover, LLMs are good at following diverse instructions, so that users are not required to follow a fixed template when asking a question. This has led to concerns about prompt sensitivity, where differences in prompt phrasing can substantially affect benchmark performance, casting doubt on the reliability of evaluations (Polo et al., 2024; Mizrahi et al., 2024; Chatterjee et al., 2024; Sclar et al., 2023). More critically, the relative rankings of LLMs can shift substantially depending on the prompt template used (Polo et al., 2024; Mizrahi et al., 2024). For example, simply changing the option format from letters (*e.g.*, "A:") to numbers (*e.g.*, "(1)") completely *reverses* the ranking order of four evaluated open-source models in ARC-Challenge (Clark et al., 2018).

Although existing studies have reported that LLMs are highly sensitive to prompt phrasing (Voronov et al., 2024; Mizrahi et al., 2024), this remains counterintuitive given that instruction-tuned LLMs are explicitly optimized to handle a wide range of input formats. For example, instruction-tuning datasets such as FLAN (Longpre et al., 2023) and Super-NaturalInstructions (Wang et al., 2022) include a diverse collection of tasks (e.g., question answering, summarization, classification) with varying natural language prompt templates (Zhang et al., 2023). This contradiction raises a critical question:

*Is prompt sensitivity an inherent flaw in LLMs, or merely an artifact of the evaluation process?*

To investigate this, we find that previous studies (Voronov et al., 2024; Chatterjee et al., 2024) typically rely on heuristic evaluation, such as regular-expression-based answer extraction or log-likelihood scoring over candidates. These heuristic evaluation approaches, though historically popular due to their simplicity (Zellers et al., 2019; Reddy et al., 2019; Brown et al., 2020; Hendrycks et al., 2021a), may introduce errors when model outputs deviate from expected formats. More specifically, models may generate correct answers, but because their outputs are not aligned with the rigid evaluation format, they are mistakenly marked as incorrect. This issue becomes more pronounced as models have become more open-ended and diverse in their output formats (Hurst et al., 2024; Yang et al., 2025), potentially leading to inflated esti-

*Equal contributions.



*Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 19889–19899
November 4-9, 2025 ©2025 Association for Computational Linguistics

[5] C04 / S04 - publisher passage
ProSA: Assessing and Understanding the Prompt Sensitivity of LLMs
Abstract. PDF page 1. Printed page 1950.
https://aclanthology.org/2024.findings-emnlp.108.pdf
Supports: ProSA studies instance-level prompt variation and reports variation across models and datasets.

# ProSA: Assessing and Understanding the Prompt Sensitivity of LLMs

**Jingming Zhuo**[1,2,*], **Songyang Zhang**[1,*], **Xinyu Fang**[1,3], **Haodong Duan**[1]
**Dahua Lin**[1,4], **Kai Chen**[1,†]
[1]Shanghai AI Laboratory, [2]Jilin University, [3]Zhejiang University,
[4]The Chinese University of Hong Kong
jingmingzhuo@gmail.com, zhangsongyang@pjlab.org.cn
* equal contribution, † corresponding author

## Abstract

Large language models (LLMs) have demonstrated impressive capabilities across various tasks, but their performance is highly sensitive to the prompts utilized. This variability poses challenges for accurate assessment and user satisfaction. Current research frequently overlooks instance-level prompt variations and their implications on subjective evaluations. To address these shortcomings, we introduce **ProSA**, a framework designed to evaluate and comprehend prompt sensitivity in LLMs. ProSA incorporates a novel sensitivity metric, `PromptSensiScore`, and leverages decoding confidence to elucidate underlying mechanisms. Our extensive study, spanning multiple tasks, uncovers that prompt sensitivity fluctuates across datasets and models, with larger models exhibiting enhanced robustness. We observe that few-shot examples can alleviate this sensitivity issue, and subjective evaluations are also susceptible to prompt sensitivities, particularly in complex, reasoning-oriented tasks. Furthermore, our findings indicate that higher model confidence correlates with increased prompt robustness. We believe this work will serve as a helpful tool in studying prompt sensitivity of LLMs. The project is released at: https://github.com/open-compass/ProSA.

## 1 Introduction

In recent years, large language models (LLMs) have rapidly become the focus of the artificial intelligence field. By training on large-scale corpora, LLMs have shown impressive capabilities in multiple tasks (Zhao et al., 2023; Min et al., 2023). The input for LLMs, known as prompts, is crucial for their ability to complete a wide variety of tasks. The prompts for LLMs come in various forms. Even for the same requirement, different individuals' varying expression habits can result in prompts of different styles. Figure 1 illustrates four styles of prompts used by LLMs when completing mathematical tasks.

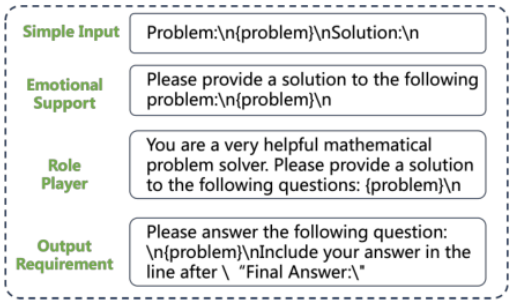



Figure 1: **A Showcase of the Four Prompt Templates on MATH.** These four prompt templates represent four different styles of constructing prompts, serving as an example of the diversity in human prompt expression.

The diversity of prompts elicits various responses from LLMs. Recent studies (Zhu et al., 2023; Pezeshkpour and Hruschka, 2024; Sclar et al., 2024; Zhou et al., 2023) investigate model performance across different prompt templates and demonstrate that LLMs are highly sensitive to the nuances of prompts. Even minor alterations to prompts can lead to substantial declines in model performance. This sensitivity to prompts poses a challenge for researchers aiming to precisely evaluate the models' capabilities. Moreover, users frequently have to iterate on their prompts numerous times to achieve higher quality outputs.

However, current research on prompt sensitivity mainly centers on dataset-level analyses, focusing on performance variations across distinct prompt templates within identical datasets. Also, existing works neglect examining how LLMs align with human expectations under different prompts in subjective evaluations, hindering the accuracy of insights reflecting real-world user experiences. Moreover, it's remain changeling in elucidating the reasons behind LLMs' sensitivity to prompts.



*Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1950–1976
November 12-16, 2024 ©2024 Association for Computational Linguistics

[6] C05 / S05 - publisher passage

POSIX: A Prompt Sensitivity Index For Large Language Models
Abstract. PDF page 1. Printed page 14550.
https://aclanthology.org/2024.findings-emnlp.852.pdf
Supports: POSIX measures response likelihood changes under prompt variation, a different endpoint from program correctness.

# POSIX: A Prompt Sensitivity Index For Large Language Models

**Anwoy Chatterjee**[*†]
Dept. of Electrical Engineering
Indian Institute of Technology Delhi
anwoy.chatterjee@ee.iitd.ac.in

**H S V N S Kowndinya Renduchintala**[†]
Media and Data Science Research
Adobe Inc., India
rharisrikowndinya333@gmail.com

**Sumit Bhatia**
Media and Data Science Research
Adobe Inc., India
sumit.bhatia@adobe.com

**Tanmoy Chakraborty**
Dept. of Electrical Engineering
Indian Institute of Technology Delhi
tanchak@iitd.ac.in

## Abstract

Despite their remarkable capabilities, Large Language Models (LLMs) are found to be surprisingly sensitive to minor variations in prompts, often generating significantly divergent outputs in response to minor variations in the prompts, such as spelling errors, alteration of wording or the prompt template. However, while assessing the quality of an LLM, the focus often tends to be solely on its performance on downstream tasks, while very little to no attention is paid to prompt sensitivity. To fill this gap, we propose POSIX – a novel ***PrOmpt Sensitivity IndeX*** as a reliable measure of prompt sensitivity, thereby offering a more comprehensive evaluation of LLM performance. The key idea behind POSIX is to capture the relative change in log-likelihood of a given response upon replacing the corresponding prompt with a different intent-preserving prompt. We provide thorough empirical evidence demonstrating the efficacy of POSIX in capturing prompt sensitivity and subsequently use it to measure and thereby compare prompt sensitivity of various open-source LLMs. We find that merely increasing the parameter count or instruction tuning does not necessarily reduce prompt sensitivity whereas adding some few-shot exemplars, even just one, almost always leads to significant decrease in prompt sensitivity. We also find that alterations to prompt template lead to the highest sensitivity in the case of MCQ-type tasks, whereas paraphrasing results in the highest sensitivity in open-ended generation tasks. The code for reproducing our results is open-sourced at https://github.com/kowndinya-renduchintala/POSIX.

## 1 Introduction

Large Language Models (LLMs) are pre-trained on enormous amounts of text data using the *next-token-prediction* objective and they can perform a variety of NLP tasks via *"prompting"* (Brown et al., 2020; Kojima et al., 2022; Almazrouei et al., 2023; Liu et al., 2023; Touvron et al., 2023). However, LLMs have been found to be surprisingly sensitive even to the smallest of variations in prompts that do not significantly alter its meaning – such as wording, prompt template or even minor spelling errors – so much so that *prompt engineering*, which is a process of iteratively tuning prompts to elicit desired responses, has become a widespread practice (Reynolds and McDonell, 2021).

Despite prompt sensitivity being a crucial aspect for assessing the usability of an LLM, standard evaluation benchmarks such as MMLU (Hendrycks et al., 2021) or BBH (Suzgun et al., 2022) focus predominantly on performance metrics like exact match, leaving prompt sensitivity sidelined. Similarly, the model cards and blog posts announcing the most commonly used LLMs often do not contain prompt sensitivity analysis at all (AI@Meta, 2024). However, from a user-centric perspective, models with low prompt sensitivity are generally preferred over highly prompt-sensitive ones, even if both perform similarly on standard benchmarks. This is largely because a real-world user may not always be able to formulate the "optimal" prompt everytime. Moreover, a universally applicable optimal prompt that can work *across* different model architectures may not even exist. Therefore, it becomes essential to develop a dedicated procedure to systematically evaluate and quantify the sensitivity of LLMs towards *intent-preserving* (or, *intent-aligned*) variations in prompts.

While the exploration of the topic of quantifying prompt sensitivity is limited, there exist a few works which study prompt sensitivity and attempt to quantify it. For instance, the HELM benchmark (Liang et al., 2023) implements various kinds of perturbations to the prompts, such as typos and misspellings, and reports the exact match score of

[†]These two authors contributed **equally** to this work.
[*]Work done during internship at Media and Data Science Research (MDSR) Lab, Adobe Inc.



*Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 14550–14565
November 12-16, 2024 ©2024 Association for Computational Linguistics

**[8] C06 / S06 - publisher passage**

Prompt Perturbation Consistency Learning for Robust Language Models
Abstract. PDF page 1. Printed page 1357.
https://aclanthology.org/2024.findings-eacl.91.pdf
Supports: The authors study perturbations in intent classification and slot filling and propose consistency learning.

# Prompt Perturbation Consistency Learning for Robust Language Models

**Yao Qiang**[1,*], **Subhrangshu Nandi**[2], **Ninareh Mehrabi**[2], **Greg Ver Steeg**[2], **Anoop Kumar**[2], **Anna Rumshisky**[2,3], and **Aram Galstyan**[2]

[1]Wayne State University, Detroit, USA
[2]Amazon
[3]University of Massachusetts Lowell
[1]yao@wayne.edu
[2]{subhrn,mninareh,gssteeg,anooamzn,arrumshi,argalsty}@amazon.com

## Abstract

Large language models (LLMs) have demonstrated impressive performance on a number of natural language processing tasks, such as question answering and text summarization. However, their performance on sequence labeling tasks, such as intent classification and slot filling (IC-SF), which is a central component in personal assistant systems, lags significantly behind discriminative models. Furthermore, there is a lack of substantive research on robustness of LLMs to various perturbations in the input prompts. The contributions of this paper are three-fold. First, we show that fine-tuning sufficiently large LLMs can produce IC-SF performance comparable to discriminative models. Next, we systematically analyze the performance deterioration of those fine-tuned models due to three distinct yet relevant types of input perturbations - oronyms, synonyms, and paraphrasing. Finally, we propose an efficient mitigation approach, *prompt perturbation consistency learning* (PPCL), which works by regularizing the divergence between losses from clean and perturbed samples. Our experiments show that PPCL can recover on an average 59% and 69% of the performance drop for IC and SF tasks, respectively. Furthermore, PPCL beats data augmentation approach while using ten times fewer augmented data samples.

## 1 Introduction

Voice controlled smart personal assistants like Amazon Echo and Google Home have flourished in recent years, enabling goal-oriented conversations and aiding tasks like setting reminders, checking weather, controlling smart devices, and online shopping. A core capability of those systems is to perform accurate and robust intent classification (IC) and slot filling (SF) (Tur and De Mori, 2011; Qin et al., 2021). The IC task involves identifying the speaker's desired intent from a given utterance, while the SF task involves recognizing the key arguments of the intent. For instance, given a user query "wake me up at five am this week.", the intent is 'set alarm', while the SF component should identify the specific details, such as 'five am' as time and 'this week' as date for the alarm setting.

Pre-trained LLMs hold promise of greatly improving personal assistant systems, owing to their impressive conversational and reasoning capabilities. In addition to generating fluent conversations, LLMs have shown SOTA performance on a variety of natural language processing (NLP) tasks such as text classification, question answering, text summarization (Chowdhery et al., 2022; Qin et al., 2023). Furthermore, some LLMs have shown promising ability to generate structured outputs such as code synthesis (Nijkamp et al., 2023) and API calls (Patil et al., 2023). However, the performance of LLMs on other structured prediction tasks such as slot filling lags significantly behind.

Another important issue is that LLMs can be highly sensitive to prompt variations (Webson and Pavlick, 2022; Min et al., 2022; Ye and Durrett, 2022). For instance, varying the order of few-shot examples, introducing minor typos or different expressions with the same semantic meaning can lead to qualitatively different results (Jin et al., 2020; Li et al., 2020; Huang et al., 2021; Zhuo et al., 2023). In conversational systems, such perturbations might be caused by automatic speech recognition (ASR) errors, linguistic differences, and user-specific expressions. Thus, adopting LLMs for voice-based personal assistants requires a good understanding of their robustness to above types of perturbations, and effective mitigation to have robust LLM-based IC-SF models.

In this paper we mainly consider the following questions: (1) How can we close the performance gap between LLMs and SOTA discriminative models on IC-SF tasks? (2) How does the performance of LLMs change due to minor changes in the origi-

[*]This work was done while interning at Amazon.



*Findings of the Association for Computational Linguistics: EACL 2024*, pages 1357–1370
March 17-22, 2024 ©2024 Association for Computational Linguistics

[7] C07 / S07 - publisher passage
How are Prompts Different in Terms of Sensitivity?
Abstract. PDF page 1. Printed page 5833.
https://aclanthology.org/2024.naacl-long.325.pdf
Supports: The authors use sensitivity as a performance proxy in their evaluated setting.

# How are Prompts Different in Terms of Sensitivity?

**Sheng Lu, Hendrik Schuff, Iryna Gurevych**

Ubiquitous Knowledge Processing Lab (UKP Lab)
Department of Computer Science and Hessian Center for AI (hessian.AI)
Technical University of Darmstadt
www.ukp.tu-darmstadt.de

## Abstract

In-context learning (ICL) has become one of the most popular learning paradigms. While there is a growing body of literature focusing on prompt engineering, there is a lack of systematic analysis comparing the effects of prompt techniques across different models and tasks. To address this, we present a comprehensive prompt analysis based on sensitivity. Our analysis reveals that sensitivity is an unsupervised proxy for model performance, as it exhibits a strong negative correlation with accuracy. We use gradient-based saliency scores to empirically demonstrate how different prompts affect the relevance of input tokens to the output, resulting in different levels of sensitivity. Furthermore, we introduce *sensitivity-aware* decoding which incorporates sensitivity estimation as a penalty term in the standard greedy decoding. We show that this approach is particularly helpful when information in the input is scarce. Our work provides a fresh perspective on the analysis of prompts, and contributes to a better understanding of the mechanism of ICL.[1]

## 1 Introduction

In-context learning (ICL) has become a popular learning paradigm in natural language processing (NLP) due to the rapid development of large language models (LLMs) (Brown et al., 2020; Dong et al., 2022; Liu et al., 2023). With carefully constructed prompts, ICL achieves impressive performance on various tasks (Kojima et al., 2022; Lampinen et al., 2022; Wei et al., 2022b; Srivastava et al., 2023). As a result, prompt engineering, which aims to find prompts that lead to optimal performance, has emerged as a crucial research topic in ICL (White et al., 2023; Zhou et al., 2023b). Although effort has been made to understand the effectiveness of certain prompt techniques (Feng et al., 2023; Gonen et al., 2023; Wang et al., 2023a), there is no systematic analysis of them across various tasks and models (Ajith et al., 2023). Such an analysis is crucial for prompt engineering, prompt selection, and gaining a deeper understanding of the working mechanism of ICL.

In this paper, we present a systematic and comprehensive analysis of prompts based on the sensitivity of a function (Hahn et al., 2021). We hypothesize that certain prompts are more effective for a given task because they decrease the level of sensitivity. Based on the recent findings that ICL implements gradient descent implicitly (Akyürek et al., 2023; Li et al., 2023; Von Oswald et al., 2023; Zhang et al., 2023), an effective prompt can be seen as one that facilitates the learning of a new function with lower sensitivity compared to the original function learnt by the model. The sensitivity of a function enables a novel framework to analyze the effect of different prompts. See Figure 1 for more details.

We did extensive experiments to validate our hypothesis. We chose five widely used natural language understanding and common sense reasoning tasks. We selected models with varying sizes from three popular families: GPT, LLaMA, and T5. We tested different prompts, including both human-designed prompts and prompts generated by an LLM. The results strongly support our hypothesis that models exhibit different levels of sensitivity depending on the prompts used, and sensitivity is an unsupervised proxy of performance as it has a strong negative correlation between accuracy and sensitivity. With the help of gradient-based saliency scores, we find that tokens in the prompt (e.g., instructions) are more relevant to the output than tokens in the input where perturbations took place, which explains how different prompts lead to varying levels of sensitivity. Furthermore, we introduce *sensitivity-aware* decoding, which incorporates sensitivity estimation as a penalty term in greedy decoding. We show that *sensitivity-aware*

[1] Our code is available at https://github.com/UKPLab/naacl2024-prompt-sensitivity.



*Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5833–5856
June 16-21, 2024 ©2024 Association for Computational Linguistics

# Calibrate Before Use: Improving Few-Shot Performance of Language Models

**Tony Z. Zhao** [*1] **Eric Wallace** [*1] **Shi Feng** [2] **Dan Klein** [1] **Sameer Singh** [3]

## Abstract

GPT-3 can perform numerous tasks when provided a natural language prompt that contains a few training examples. We show that this type of few-shot learning can be unstable: the choice of prompt format, training examples, and even the order of the training examples can cause accuracy to vary from near chance to near state-of-the-art. We demonstrate that this instability arises from the bias of language models towards predicting certain answers, e.g., those that are placed near the end of the prompt or are common in the pre-training data. To mitigate this, we first estimate the model's bias towards each answer by asking for its prediction when given the training prompt and a content-free test input such as "N/A". We then fit calibration parameters that cause the prediction for this input to be uniform across answers. On a diverse set of tasks, this *contextual calibration* procedure substantially improves GPT-3 and GPT-2's average accuracy (up to 30.0% absolute) and reduces variance across different choices of the prompt.

## 1. Introduction

Few-shot learning—the ability to learn tasks with limited examples—is an important aspect of intelligence (Lake et al., 2015; Yogatama et al., 2019). Recent work shows that large neural language models can perform few-shot learning without finetuning (Radford et al., 2019; Brown et al., 2020). Specifically, GPT-3 (Brown et al., 2020) can perform numerous tasks when provided a few examples in a natural language *prompt*. For example, to perform sentiment analysis one can condition GPT-3 on a prompt such as:

```
Input: Subpar acting. Sentiment: Negative
Input: Beautiful film. Sentiment: Positive
Input: Amazing.       Sentiment:
```

where the first two lines correspond to two training examples and the last line is a test example. To make predictions, the model predicts whether the subsequent token is more likely to be the word "Positive" or "Negative".

This style of few-shot "in-context" learning is interesting because it shows that the model can learn without parameter updates. And, more importantly, it has numerous practical advantages over the now-standard approach of finetuning (Radford et al., 2018; Devlin et al., 2019). First, it allows practitioners to "rapidly prototype" NLP models: changing the prompt *immediately* leads to a new model. Second, it provides a fully natural language interface to a machine learning model, which allows users—even those without technical expertise—to create NLP systems. Finally, since in-context learning reuses the same model for each task, it reduces memory requirements and system complexity when serving many different tasks.

However, despite these promises, we show that GPT-3's accuracy can be highly unstable across different prompts (Section 3). A prompt contains three components: a format, a set of training examples, and a permutation (ordering) for those examples. We show that different choices for these factors can lead to highly different accuracies, e.g., changing the permutation of the training examples in a sentiment analysis prompt can change accuracy from near chance (54%) to near state-of-the-art (93%). This instability implies that GPT-3 users, who typically design prompts manually, cannot expect to consistently obtain good accuracy.

We next analyze what causes this instability. We identify three pitfalls of language models that lead them to be biased toward certain answers during few-shot learning. In particular, they suffer from majority label bias, recency bias, and common token bias (Section 4). The majority label and recency biases lead the model to predict training answers that appear frequently or near the end of the prompt. For example, a prompt that ends with a Negative training example may cause a bias towards the Negative class. On the other hand, the common token bias leads the model to prefer answers that are frequent in its pre-training data, e.g.,

[*]Equal contribution [1]UC Berkeley [2]University of Maryland [3]UC Irvine. Correspondence to: Eric Wallace <ericwallace@berkeley.edu>.

*Proceedings of the 38$^{th}$ International Conference on Machine Learning*, PMLR 139, 2021. Copyright 2021 by the author(s).

**[9] C09 / S09 - publisher passage**

On the Worst Prompt Performance of Large Language Models
Abstract. PDF page 1. Printed page 1 (conference PDF pagination).
https://proceedings.neurips.cc/paper_files/paper/2024/file/7fa5a377b7ffabcce43cd00231bb3f9c-Paper-Conference.pdf
Supports: RobustAlpacaEval studies semantically equivalent case-level queries, a different intervention from fixed formats.

# On the Worst Prompt Performance of Large Language Models*

**Bowen Cao**[♢,†] **Deng Cai**[♡,‡] **Zhisong Zhang**[♡] **Yuexian Zou**[♠] **Wai Lam**[♢]
♢ The Chinese University of Hong Kong
♡ Tencent AI Lab ♠ Peking University
bwcao@link.cuhk.edu.hk, thisisjcykcd@gmail.com, zhisonzhang@tencent.com
zouyx@pku.edu.cn, wlam@se.cuhk.edu.hk

## Abstract

The performance of large language models (LLMs) is acutely sensitive to the phrasing of prompts, which raises significant concerns about their reliability in real-world scenarios. Existing studies often divide prompts into task-level instructions and case-level inputs and primarily focus on evaluating and improving robustness against variations in tasks-level instructions. However, this setup fails to fully address the diversity of real-world user queries and assumes the existence of task-specific datasets. To address these limitations, we introduce RobustAlpacaEval, a new benchmark that consists of semantically equivalent case-level queries and emphasizes the importance of using the worst prompt performance to gauge the lower bound of model performance. Extensive experiments on RobustAlpacaEval with ChatGPT and six open-source LLMs from the Llama, Mistral, and Gemma families uncover substantial variability in model performance; for instance, a difference of 45.48% between the worst and best performance for the Llama-2-70B-chat model, with its worst performance dipping as low as 9.38%. We further illustrate the difficulty in identifying the worst prompt from both model-agnostic and model-dependent perspectives, emphasizing the absence of a shortcut to characterize the worst prompt. We also attempt to enhance the worst prompt performance using existing prompt engineering and prompt consistency methods, but find that their impact is limited. These findings underscore the need to create more resilient LLMs that can maintain high performance across diverse prompts. Data and code are available at https://github.com/bwcao/RobustAlpacaEval.

## 1 Introduction

In recent years, large language models (LLMs) have made extraordinary progress (Brown et al., 2020; Touvron et al., 2023; Jiang et al., 2023; Team et al., 2024) and one key factor in their success is their ability to adapt to diverse tasks through prompting. However, the performance of these LLMs is significantly sensitive to the prompts they receive (Gu et al., 2022; Mizrahi et al., 2024; Sun et al., 2023; Li et al., 2024). Even minor alterations in format, without semantic changes, can trigger substantial performance degradation (Sclar et al., 2023). Consequently, prompt engineering has surfaced as a critical component, playing a critical role in unlocking the full potential of these models (Shin et al., 2020; Zhou et al., 2022a; Pryzant et al., 2023; Prasad et al., 2023; Gonen et al., 2022; Schulhoff et al., 2024).

*The work described in this paper is partially supported by a grant from the Research Grant Council of the Hong Kong Special Administrative Region, China (Project Code: 14200719).

†Work done during an internship at Tencent AI Lab.

‡Corresponding author.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

[10] C10 / S10 - publisher passage
Same Question, Different Words: A Latent Adversarial Framework for Prompt Robustness
Abstract. PDF page 1. Printed page 31305.
https://aclanthology.org/2025.emnlp-main.1595.pdf
Supports: Latent Adversarial Paraphrasing optimizes trainable latent perturbations and model performance.

# Same Question, Different Words: A Latent Adversarial Framework for Prompt Robustness

**Tingchen Fu**
Renmin University of China
lucas.futingchen@gmail.com

**Fazl Barez**
University of Oxford
WhiteBox & Martian
fazl@robots.ox.ac.uk

## Abstract

Insensitivity to semantically-preserving variations of prompts (paraphrases) is crucial for reliable behavior and real-world deployment of large language models. However, language models exhibit significant performance degradation with semantically equivalent but differently phrased prompts, and existing solutions either depend on trial-and-error prompt engineering or require computationally expensive inference-time algorithms. In this study, built on the key insight that worst-case prompts exhibit a drift in embedding space, we present **L**atent **A**dversarial **P**araphrasing (**LAP**), a dual-loop adversarial framework that optimizes a trainable perturbation as "latent continuous paraphrase" and language model performance on these perturbations iteratively. Extensive experiments are conducted to demonstrate the effectiveness of **LAP** across multiple backbones on the RobustAlpaca benchmark with a $0.5\% \sim 4\%$ absolution improvement on worst-case win-rate.

## 1 Introduction

Large language models (LLMs) (OpenAI, 2023; Dubey et al., 2024; Reid et al., 2024; Team, 2024) have demonstrated remarkable capabilities across diverse applications (Rozière et al., 2023; Azerbayev et al., 2023). However, these models exhibit a critical sensitivity to semantically-preserving variations in prompts (Cao et al., 2024). Our analysis using Llama-2-13b-chat (Touvron et al., 2023) on RobustAlpaca (Cao et al., 2024) reveals that the best-case reward can be twice as large as the worst-case reward for semantically equivalent prompts, as shown in Figure 1, which poses a significant challenge for real-world applications where consistent performance is crucial.

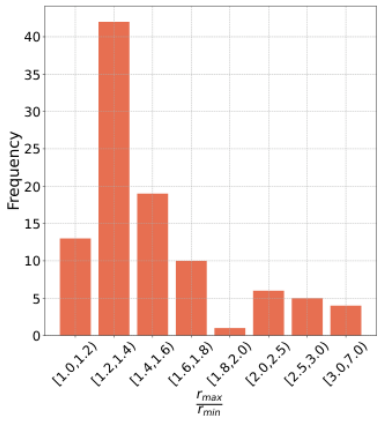



Figure 1: Distribution of the ratio between the highest and lowest reward scores (the highest and lowest reward among model response to different paraphrases of user queries) for Llama-2-13b-chat on RobustAlpaca. A higher ratio indicates greater performance variability across semantically equivalent paraphrases.

Previous approaches have attempted to address this problem by searching for an optimal prompt with prompt engineering and template optimization (Shin et al., 2020; Prasad et al., 2023; Zhou et al., 2023). However, these solutions present significant limitations in practical applications, and it is infeasible to expect users to master the intricacies of prompt design, which contradicts the goal of making LLMs accessible and reliable. Meanwhile, a concurrent line of work adopts inference-time algorithms to improve response quality through prompt rewriting (Cao et al., 2024) or response revision (Anonymous, 2025), which introduce substantial computational overhead and extra inference latency. Therefore, in this study, we take a different pathway and aim to optimize LLM on the worst-case paraphrasing to enhance the inherent robustness of LLM to prompt variation, especially the paraphrasing of the user query.

However, identifying worst-case paraphrasing


*Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 31305–31319
November 4-9, 2025 ©2025 Association for Computational Linguistics

## [11] C11 / S11 - official dataset record

BAAI/TACO, pinned Hugging Face dataset card

Not a peer-reviewed paper; additional dataset attribution.

Accessed 23 October 2026. Browser screenshot, highlighted description.

https://huggingface.co/datasets/BAAI/TACO/blob/d593ed0a2becbbc952230bb89be09189bf1056dc/README.md

Supports: The official dataset record describes code generation from natural-language specifications.

